

NumPy Tutorial 7: Boolean Indexing — Filtering Data with Conditions

This chapter covers Boolean indexing in NumPy. After completing this tutorial, students will understand how to filter data from arrays using conditions — a very important skill in Data Science.

1. Introduction

In previous tutorials, we learned how to access data using index numbers. In this tutorial, we will learn a more powerful way — **Boolean indexing**.

Boolean indexing allows us to filter data based on conditions. For example, find all students who scored more than 70 marks, or find all temperatures above 30 degrees.

This is one of the most commonly used techniques in real world Data Science.

2. What is Boolean Indexing

Boolean indexing means using True and False values to select elements from an array.

When we apply a condition to an array, NumPy creates a boolean array — an array of True and False values. We then use this boolean array to filter the original data.

3. Creating a Boolean Array

```
import numpy as np

marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

condition = marks > 70
print(condition)
```

Output:

```
[False  True False  True False  True  True False  True False]
```

In this example:

- For every element, NumPy checks if it is greater than 70.

- If yes — True. If no — False.
- The result is a boolean array with the same shape as the original.

4. Filtering Data Using Boolean Array

Now we use this boolean array to get only the elements where the condition is True.

```
import numpy as np

marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

passed = marks[marks > 70]
print(passed)
```

Output:

```
[72 90 88 76 95]
```

Only the values where the condition is True are returned. This is Boolean indexing in one line.

5. Multiple Conditions

We can combine multiple conditions using & (AND) and | (OR).

AND condition — both conditions must be True

```
marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

result = marks[(marks > 60) & (marks < 85)]
print(result)
```

Output:

```
[72 68 76]
```

Only values greater than 60 AND less than 85 are returned.

OR condition — at least one condition must be True

```
result = marks[(marks < 50) | (marks > 90)]  
print(result)
```

Output:

```
[45 95]
```

Values less than 50 OR greater than 90 are returned.

6. Using np.where

np.where is a powerful function that works like an if-else condition for arrays.

Format: np.where(condition, value_if_true, value_if_false)

```
import numpy as np  
  
marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])  
  
result = np.where(marks >= 70, "Pass", "Fail")  
print(result)
```

Output:

```
['Fail' 'Pass' 'Fail' 'Pass' 'Fail' 'Pass' 'Pass' 'Fail' 'Pass' 'Fail']
```

In this example:

- If marks is greater than or equal to 70 — return "Pass".
 - Otherwise — return "Fail".
 - The result is a new array with Pass and Fail labels.
-

7. np.where to Replace Values

We can also use np.where to replace values in an array.

```
arr = np.array([10, -5, 20, -8, 15, -3])
```

```
result = np.where(arr < 0, 0, arr)
print(result)
```

Output:

```
[10  0 20  0 15  0]
```

All negative values are replaced with 0. Positive values remain the same.

8. Counting Elements That Match a Condition

```
marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

passed_count = np.sum(marks >= 70)
print("Students who passed:", passed_count)

failed_count = np.sum(marks < 70)
print("Students who failed:", failed_count)
```

Output:

```
Students who passed: 5
Students who failed: 5
```

`np.sum` counts how many True values are in the boolean array — because True = 1 and False = 0 in NumPy.

9. Boolean Indexing on 2D Arrays

```
import numpy as np

scores = np.array([[85, 45, 78],
                  [92, 38, 55],
                  [70, 75, 30]])

print(scores[scores < 50])
```

Output:

```
[45 38 55 30]
```

NumPy flattens the result when filtering a 2D array with a condition. All elements less than 50 are returned as a 1D array.

10. Practical Example — Student Grade Analysis

```
import numpy as np

names = np.array(["Ali", "Sara", "Ahmed", "Zara", "Umar"])
marks = np.array([85, 42, 78, 55, 90])

top_students = names[marks >= 75]
print("Top students:", top_students)

failed_students = names[marks < 50]
print("Failed students:", failed_students)
```

Output:

```
Top students: ['Ali' 'Ahmed' 'Umar']
Failed students: ['Sara']
```

We used the boolean array created from marks to index the names array. This is a very common pattern in Data Science.

11. Conclusion of Tutorial 7

In this tutorial, students have learned:

1. What Boolean indexing is and how it works.
2. How to create a boolean array using conditions.
3. How to filter data using boolean indexing.
4. How to use multiple conditions with AND and OR.
5. How to use np.where for if-else operations on arrays.
6. How to count elements matching a condition.
7. How to apply boolean indexing on 2D arrays.
8. A practical example of filtering student data.

In the next tutorial, we will learn about random number generation in NumPy.

This completes NumPy Tutorial 7.